



ÉCOLE NATIONALE SUPÉRIEURE
D'INFORMATIQUE ET D'ANALYSE DES SYSTÈMES
- RABAT

PROJET MLOPS

Détection de la fraude dans les
transactions par carte de crédit

Élèves :

Mouhsine ELAOULA
Soufiane ELOULJI
Aymane AMRAOUI

Encadrant :

Pr. Mahmoud
ELHAMLAOUI

Année Universitaire 2025-2026

Résumé

Ce projet s'inscrit dans le cadre de l'industrialisation des modèles d'apprentissage automatique (MLOps). Il vise à transformer un modèle expérimental de détection de transactions frauduleuses en un service de production robuste, scalable et automatisé. Pour répondre aux exigences de fiabilité et de performance, nous avons développé une API REST performante avec FastAPI, encapsulant un pipeline de Machine Learning complet. L'application a ensuite été conteneurisée via Docker pour garantir la portabilité de l'environnement. Afin d'accélérer le cycle de développement et de réduire les erreurs humaines, nous avons mis en place une chaîne d'intégration et de déploiement continus (CI/CD) avec GitHub Actions, assurant l'exécution automatique des tests unitaires et la publication des images sur Docker Hub. Enfin, l'orchestration du service a été réalisée sur un cluster Kubernetes (Minikube), assurant la haute disponibilité et la gestion dynamique de la configuration.

Mots-clés : MLOps, Détection de Fraude, FastAPI, Docker, Kubernetes, CI/CD, GitHub Actions.

Abstract

This project focuses on the industrialization of machine learning models (MLOps). It aims to transform an experimental fraud detection model into a robust, scalable, and automated production service. To meet reliability and performance requirements, we developed a high-performance REST API using FastAPI, encapsulating a complete Machine Learning pipeline. The application was then containerized using Docker to ensure environment portability. To accelerate the development cycle and minimize human error, we implemented a Continuous Integration and Continuous Deployment (CI/CD) pipeline via GitHub Actions, automating unit tests and image publishing to Docker Hub. Finally, the service orchestration was carried out on a Kubernetes cluster (Minikube), ensuring high availability and dynamic configuration management.

Keywords : MLOps, Fraud Detection, FastAPI, Docker, Kubernetes, CI/CD, GitHub Actions.

Introduction générale

La croissance rapide des transactions numériques s'est accompagnée d'une augmentation significative des tentatives de fraude, imposant aux institutions financières de se doter de systèmes de détection de plus en plus sophistiqués. Si l'Intelligence Artificielle offre des performances remarquables pour identifier ces menaces, un modèle de Machine Learning ne crée de la valeur que lorsqu'il est déployé efficacement en production.

Pourtant, un fossé technique sépare souvent l'environnement expérimental du Data Scientist (Notebooks) des exigences opérationnelles (stabilité, scalabilité, sécurité). C'est pour combler ce vide qu'est née la discipline du MLOps (Machine Learning Operations).

L'objectif de ce projet est de concevoir et de mettre en œuvre une chaîne MLOps pour un système de détection de fraude bancaire. Il ne s'agit pas seulement d'entraîner un modèle performant, mais de construire l'infrastructure logicielle permettant de le servir, de le tester et de le mettre à jour sans interruption de service.

Ce rapport détaille les étapes techniques de cette industrialisation :

1. La transformation du modèle en un microservice web via FastAPI.
2. La standardisation de l'environnement d'exécution grâce à la conteneurisation Docker.
3. L'automatisation du cycle de vie logiciel (tests et livraison) via un pipeline CI/CD sur GitHub.
4. Le déploiement orchestré et résilient sur un cluster Kubernetes.

Table des matières

Résumé	1
Abstract	2
Introduction générale	3
1 Introduction et Contexte	6
1.1 Contexte du Projet	6
1.2 Problématique MLOps	6
1.3 Objectifs Réalisés	6
1.4 Architecture et Stack Technique	7
1.5 Étapes du déploiement du service	8
2 Développement et Validation du Service	9
2.1 Architecture du Microservice (FastAPI)	9
2.2 Intégration du Pipeline de Machine Learning	9
2.3 Gestion Dynamique de la Configuration	10
2.4 Tests Automatisés	11
3 Conteneurisation et Automatisation du Déploiement (CI/CD)	12
3.1 Stratégie de Conteneurisation (Docker)	12
3.2 Pipeline d'Intégration et de Livraison Continues (CI/CD)	13
3.2.1 Intégration Continue (CI)	13
3.2.2 Livraison Continue (CD - Build & Push)	13
4 Orchestration avec Kubernetes	15
4.1 Définition de l'Architecture	15
4.1.1 Le Deployment	15
4.1.2 Le Service	16
4.2 Gestion Externalisée de la Configuration	17
4.3 Stratégie de Mise à Jour (Rolling Update)	17
5 Validation et Démonstration Fonctionnelle	19
5.1 Accès à l'Interface de Documentation (Swagger UI)	19
5.2 Test de Prédiction	19
Conclusion Générale	21
Bibliographie	22

Table des figures

1.1	Pipeline de déploiement progressif du service	8
3.1	Exécution réussie des étapes du pipeline CI	13
3.2	L'image soufianeeloulji/fraud_detection mise à jour automatiquement sur le registre Docker Hub	14
4.1	Visualisation des 3 Pods actifs (Running)	16
4.2	Obtention de l'URL d'accès au service via Minikube	17
4.3	Démonstration du Rolling Update	18
5.1	Interface listant les endpoints disponibles	19
5.2	Entrée de Test pour l'API	20
5.3	Réponse de l'API	20

Chapitre 1

Introduction et Contexte

1.1 Contexte du Projet

Dans le secteur financier et bancaire actuel, la détection des transactions frauduleuses est devenue un enjeu critique. Avec l'augmentation exponentielle des paiements numériques, les méthodes de surveillance manuelles sont obsolètes. L'intelligence artificielle, et plus particulièrement le Machine Learning, offre une solution puissante pour identifier des schémas suspects en temps réel.

Cependant, la valeur d'un modèle de Machine Learning ne se concrétise réellement que lorsqu'il sort de l'environnement de développement (souvent un Notebook Jupyter) pour être intégré dans un système de production capable de traiter de vraies requêtes. C'est dans ce cadre que s'inscrit notre projet : transformer un modèle de détection de fraude statique en un service API robuste et évolutif.

1.2 Problématique MLOps

Le passage d'un modèle expérimental à un service en production soulève plusieurs défis techniques majeurs, souvent résumés par le terme MLOps (Machine Learning Operations) :

- **Reproductibilité** : Comment garantir que le modèle fonctionne de la même manière sur la machine du Data Scientist et sur le serveur de production ?
- **Fiabilité** : Comment s'assurer qu'une mise à jour du code ou du modèle ne va pas casser le service existant ?
- **Scalabilité** : Comment gérer une augmentation de la charge sans interruption de service ?
- **Automatisation** : Comment éviter les tâches de déploiement manuelles, sources d'erreurs et de perte de temps ?

Ce projet a pour but de répondre à ces problématiques en mettant en place une chaîne d'automatisation complète (CI/CD) et une architecture conteneurisée.

1.3 Objectifs Réalisés

L'objectif principal de ce travail a été de concevoir et déployer une infrastructure MLOps de bout en bout. Les objectifs spécifiques atteints sont les suivants :

1. **Industrialisation du modèle** : Encapsulation du modèle Scikit-Learn et de ses pipelines de prétraitement dans une API REST performante.
2. **Assurance Qualité (QA)** : Mise en place de tests unitaires et d'intégration automatisés pour valider chaque modification du code.

3. **Conteneurisation** : Utilisation de Docker pour isoler l'environnement d'exécution et ses dépendances.
4. **Intégration et Déploiement Continu (CI/CD)** : Création d'un pipeline GitHub Actions automatisant les tests, la construction de l'image et son envoi vers un registre distant.
5. **Orchestration** : Déploiement de l'application sur un cluster Kubernetes (Minikube) assurant la haute disponibilité et la gestion de la configuration.

1.4 Architecture et Stack Technique

Pour répondre à ces exigences, nous avons sélectionné des technologies standards de l'industrie :

- **Langage & Framework** : Python avec FastAPI pour la création du microservice de prédiction. FastAPI a été choisi pour sa rapidité d'exécution (Asynchronous), sa validation automatique des données via Pydantic et sa documentation native (Swagger UI).
- **Machine Learning** : Scikit-learn et Pandas pour le traitement des données et l'inférence du modèle.
- **Conteneurisation** : Docker pour la création d'images légères et portables.
- **CI/CD** : GitHub Actions pour l'orchestration du pipeline d'automatisation.
- **Registre d'images** : Docker Hub pour le stockage sécurisé et le versionnage des images Docker.
- **Orchestration** : Kubernetes (via Minikube) pour le déploiement, la gestion des réplicas (Pods) et l'exposition du service.

1.5 Étapes du déploiement du service

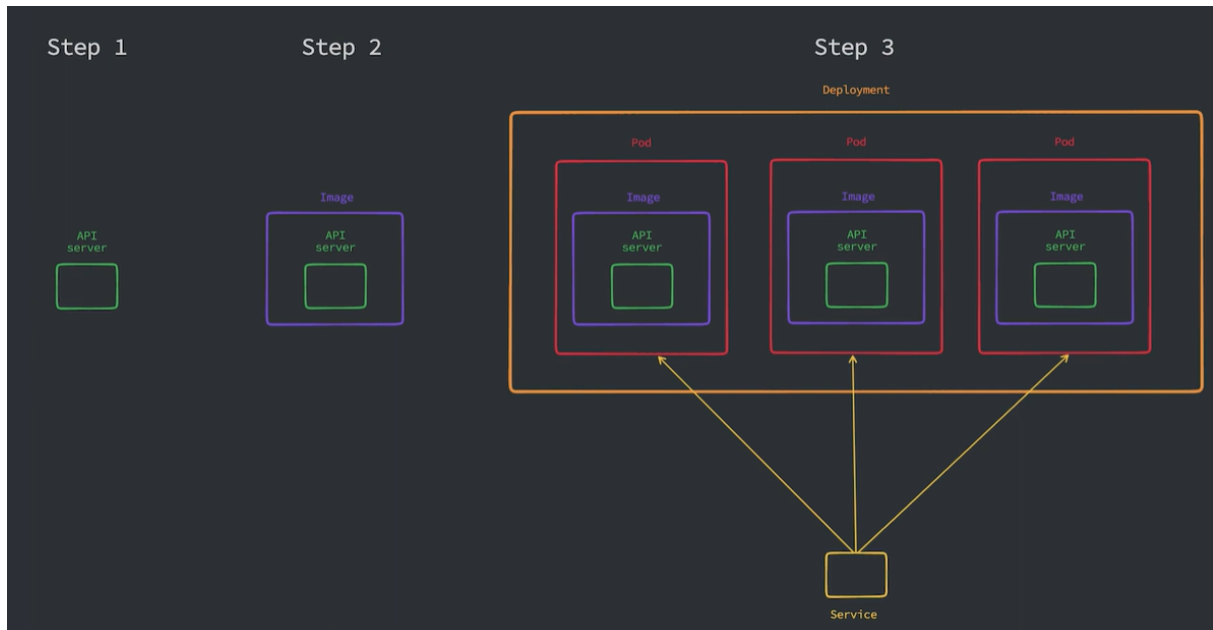


FIGURE 1.1 – Pipeline de déploiement progressif du service

Ce schéma illustre l'évolution architecturale de notre solution en trois phases clés :

1. **Développement (Step 1) :** Le cœur du système est le serveur FastAPI (en vert), qui expose le modèle de Machine Learning via une interface REST. Il est d'abord testé isolément.
2. **Conteneurisation (Step 2) :** L'application est encapsulée dans une Image Docker (en violet, `soufiane/fraud_detection`). Cette étape verrouille l'environnement (OS, librairies Python) pour garantir une portabilité totale ("Build Once, Run Anywhere").
3. **Orchestration (Step 3) :** L'image est déployée sur le cluster Kubernetes au sein d'un Deployment (cadre orange). Celui-ci maintient 3 Pods (cadres rouges) actifs en parallèle pour assurer la haute disponibilité et la tolérance aux pannes. Un objet Service (en jaune) agit comme un point d'entrée unique et répartiteur de charge (Load Balancer), distribuant les requêtes des utilisateurs vers les différentes instances disponibles.

Chapitre 2

Développement et Validation du Service

Cette section détaille la transformation de notre modèle de Machine Learning en un microservice web performant, ainsi que la stratégie de tests mise en place pour garantir sa fiabilité avant le déploiement.

2.1 Architecture du Microservice (FastAPI)

Pour exposer notre modèle de détection de fraude, nous avons choisi le framework FastAPI. Contrairement aux frameworks traditionnels (comme Flask), FastAPI repose sur le standard ASGI (Asynchronous Server Gateway Interface), offrant des performances élevées indispensables pour le traitement de transactions en temps réel.

L'architecture du service repose sur trois piliers :

1. **Typage fort des données (Pydantic)** : Chaque requête entrante est validée par un schéma strict. Si une transaction ne contient pas les champs requis (ex : amt, lat, long) ou si les types sont incorrects, l'API rejette immédiatement la requête (Erreur 422), protégeant ainsi le modèle contre des données corrompues.
2. **Documentation Autogénérée** : L'utilisation des standards OpenAPI permet de générer automatiquement une interface Swagger UI (/docs), facilitant l'interaction et le test des endpoints sans outil externe.
3. **Inférence en Temps Réel** : Le service expose un endpoint POST /predict qui reçoit les données JSON, les traite et retourne la probabilité de fraude.

2.2 Intégration du Pipeline de Machine Learning

L'un des défis majeurs du MLOps est de garantir que les données brutes envoyées à l'API subissent exactement les mêmes transformations que celles utilisées lors de l'entraînement. Pour résoudre ce problème, nous n'avons pas exporté uniquement le modèle mathématique, mais un Pipeline Scikit-Learn complet.

```
1 pipeline = joblib.load('app/model_ensemble.joblib')
2
3 app = FastAPI()
4
5 FRAUD_THRESHOLD = float(os.getenv("FRAUD_THRESHOLD", "0.75"))
6
7 @app.get('/')
8 def root():
9     return {"message": 'Fraud Detection Model'}
10
11 @app.post('/predict')
12 def predict(data: dict):
13     df_input = pd.DataFrame([data])
14     prediction = pipeline.predict_proba(df_input)[0][1]
15     is_fraud_result = bool(prediction > FRAUD_THRESHOLD)
16     return {
17         "is_fraud": is_fraud_result,
18         "fraud_probability": float(prediction),
19         "threshold_used": FRAUD_THRESHOLD
20     }
```

Listing 1 – Code de l'API - Fichier app/app.py

Le fichier `model_ensemble.joblib` encapsule une chaîne de traitement composée de deux étapes séquentielles :

1. **Le Préprocesseur (preprocessor)** : Un `ColumnTransformer` qui orchestre nos transformateurs personnalisés (`DateFeatureExtractor` pour les caractéristiques temporelles et `FrequencyEncoder` pour les variables catégorielles). Il transforme le JSON brut en vecteur numérique.
2. **Le Modèle (model)** : Un algorithme d'Ensemble Learning pré-entraîné qui reçoit les données vectorisées et effectue la classification finale.

Cette encapsulation permet à l'API de recevoir des données brutes et de gérer toute la complexité du traitement en interne. Pour assurer le chargement de ce pipeline sans erreur de sérialisation (`AttributeError`), nous avons rigoureusement séparé la définition des classes (dans `app/preprocessing.py`) de la logique d'exécution, permettant à l'application de reconstruire l'objet Python complet dès son démarrage.

2.3 Gestion Dynamique de la Configuration

Nous avons strictement séparé la configuration du code source. Le seuil de décision du modèle (le niveau de probabilité à partir duquel une transaction est jugée frauduleuse) n'est pas codé en dur.

Il est récupéré via une variable d'environnement :

```
FRAUD_THRESHOLD = float(os.getenv("FRAUD_THRESHOLD", "0.75"))
```

Cette approche permet, comme nous le verrons dans la partie déploiement, de modifier la sensibilité du modèle directement depuis l'orchestrateur (Kubernetes) sans avoir à redéploier ou modifier le code de l'application.

2.4 Tests Automatisés

La mise en production d'un modèle sans filet de sécurité est risquée. Nous avons donc implémenté une suite de tests automatisés avec Pytest.

Nous utilisons TestClient de FastAPI pour simuler des requêtes HTTP réelles sans avoir besoin de lancer le serveur. Notre couverture de tests inclut :

1. **Tests de Santé (Health Check)** : Vérification que l'API démarre et répond correctement sur la racine.

```
1 def test_read_main():
2     response = client.get("/")
3     assert response.status_code == 200
4     assert response.json() == {"message": 'Fraud Detection Model'}
```

Listing 2 – Code des tests - Fichier tests/test_{main}.py

2. **Tests de Prédiction Valide** : Envoi d'un payload JSON conforme (transaction normale et frauduleuse) pour vérifier que le modèle retourne bien une structure JSON contenant `is_fraud`, `fraud_probability` et `threshold_used`.

```
1 def test_prediction_endpoint():
2     data = {
3         #exemple d'entrée
4         }
5     response = client.post("/predict", json=data)
6     assert response.status_code == 200
7
8     data = response.json()
9     assert "is_fraud" in data
10    assert "fraud_probability" in data
11    assert isinstance(data["is_fraud"], bool)
```

Listing 3 – Code des tests - Fichier tests/test_main.py

Ces tests constituent la porte de qualité de notre pipeline : si un seul test échoue, le processus de déploiement est immédiatement stoppé.

Chapitre 3

Conteneurisation et Automatisation du Déploiement (CI/CD)

Une fois le microservice de prédiction validé fonctionnellement, l'étape suivante du processus MLOps consiste à standardiser son environnement d'exécution et à automatiser sa livraison. Cette section détaille comment nous garantissons la portabilité de l'application via Docker et son déploiement continu via GitHub Actions.

3.1 Stratégie de Conteneurisation (Docker)

Afin de résoudre les problèmes de compatibilité entre les environnements de développement et de production, nous avons encapsulé l'API, le modèle pré-entraîné et toutes les dépendances système dans un conteneur isolé.

Le fichier Dockerfile utilisé pour construire notre image a été optimisé pour la performance et la mise en cache :

```
1 FROM python:3.12
2 WORKDIR /code
3 COPY ./app/requirements.txt /code/requirements.txt
4 RUN pip install --no-cache-dir -r /code/requirements.txt
5 COPY ./app /code/app
6 EXPOSE 8000
7 CMD ["uvicorn", "app.app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Listing 4 – Dockerfile utilisé

Les choix techniques effectués dans ce fichier sont les suivants :

- **Image de Base (FROM python:3.12)** : Nous utilisons une version stable et récente de Python pour garantir la compatibilité avec les dernières bibliothèques de Data Science.
- **Optimisation du Cache (COPY requirements.txt)** : Nous copions et installons les dépendances avant de copier le code source (COPY ./app). Cette stratégie permet à Docker de mettre en cache la couche des librairies. Ainsi, si nous modifions seulement le code de l'application sans toucher aux dépendances, la reconstruction de l'image est quasi-instantanée.
- **Exécution (CMD)** : Le conteneur lance directement le serveur de production Uvicorn sur l'interface 0.0.0.0, rendant l'API accessible depuis l'extérieur du conteneur sur le port 8000.

3.2 Pipeline d'Intégration et de Livraison Continues (CI/CD)

L'automatisation est le pilier central de notre approche MLOps. Nous avons mis en place un pipeline GitHub Actions (`.github/workflows/ci.yml`) qui se déclenche automatiquement à chaque modification du code (événement push sur la branche main). Ce pipeline assure que seule une version stable et testée de l'application est déployée.

Le workflow se décompose en deux jobs séquentiels :

3.2.1 Intégration Continue (CI)

Ce premier job est une barrière de qualité. Il installe l'environnement, les dépendances et exécute la suite de tests unitaires définie précédemment.

Sécurité : Si un test échoue, le pipeline s'arrête immédiatement. Cela empêche le déploiement d'un modèle régressé ou d'une API défectueuse.

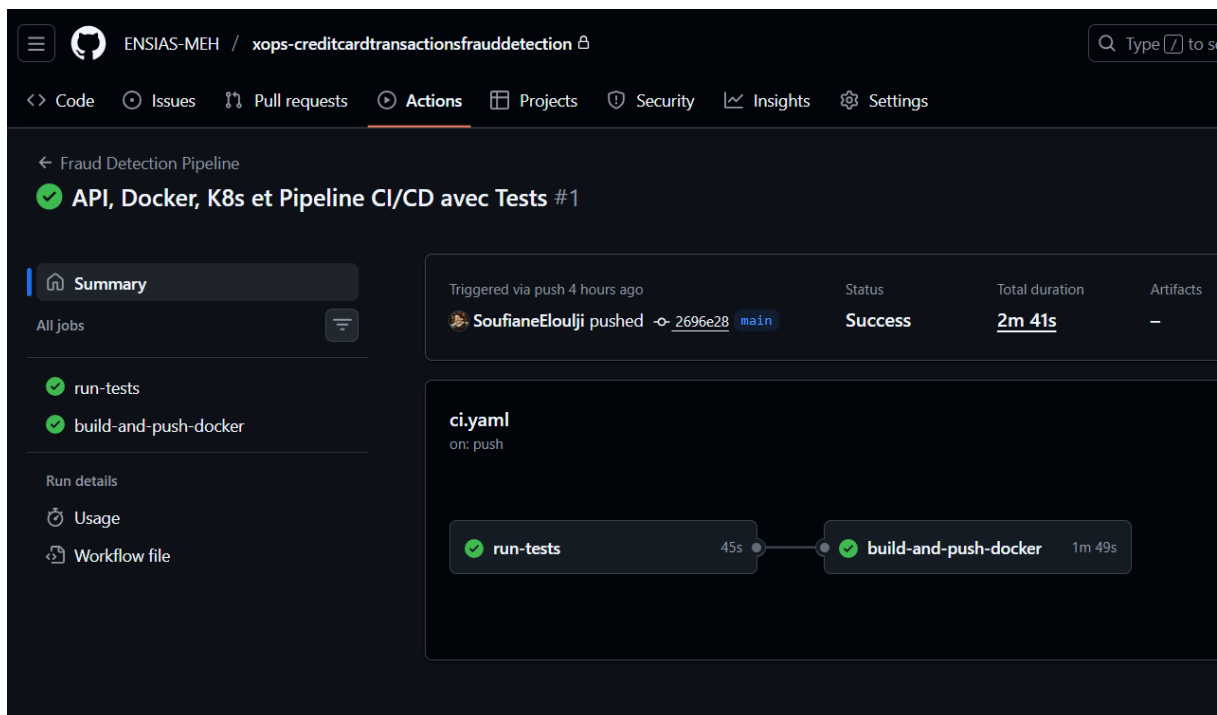


FIGURE 3.1 – Exécution réussie des étapes du pipeline CI

3.2.2 Livraison Continue (CD - Build & Push)

Une fois les tests validés, le job de livraison prend le relais pour publier l'artefact :

1. **Authentification Sécurisée** : Le pipeline se connecte au registre Docker Hub en utilisant des identifiants chiffrés stockés dans les GitHub Secrets (`DOCKERHUB_USERNAME` et `DOCKERHUB_TOKEN`). Cette pratique est essentielle pour ne jamais exposer de mots de passe dans le code source.
2. **Build et Push** : L'image est construite puis envoyée vers le dépôt public. Elle est taguée de deux manières :

- `latest` : Pour indiquer qu’il s’agit de la version de production actuelle.
- SHA du commit (hash) : Pour garantir la traçabilité et permettre un retour en arrière (rollback) vers une version spécifique si nécessaire.

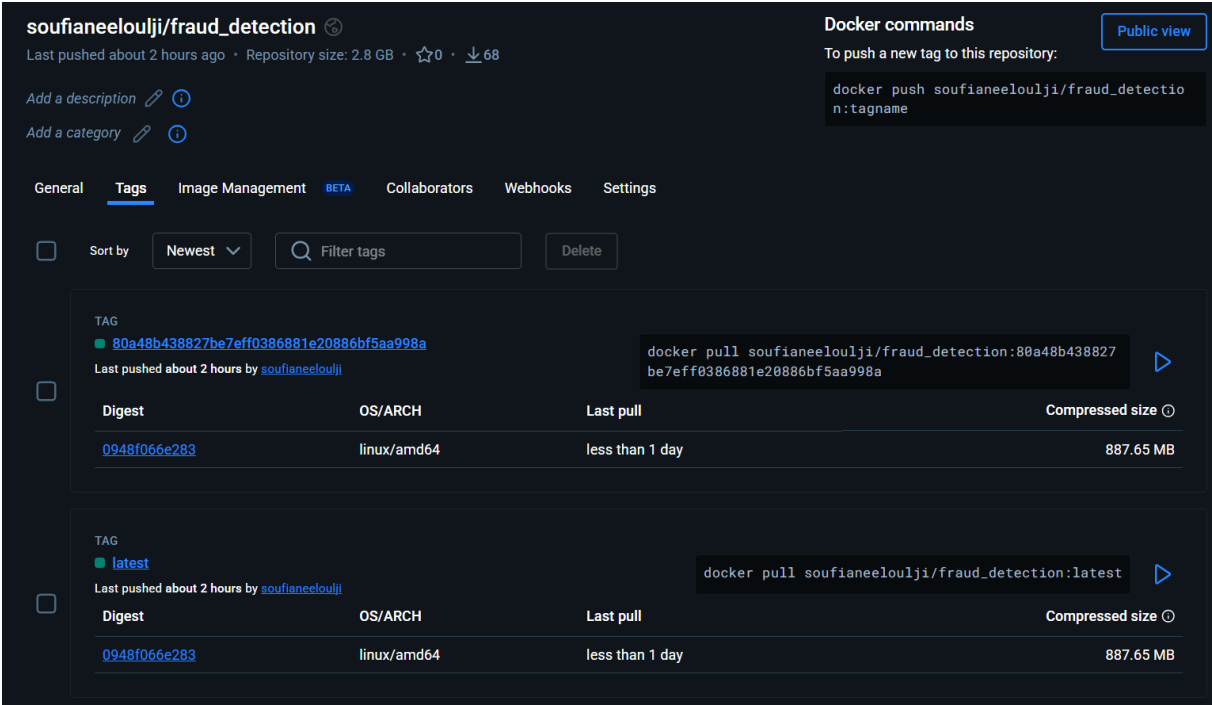


FIGURE 3.2 – L’image `soufianeeloulji/fraud_detection` mise à jour automatiquement sur le registre Docker Hub

Chapitre 4

Orchestration avec Kubernetes

Si Docker permet de créer des conteneurs isolés, il ne suffit pas pour gérer une application en production (gestion des pannes, mise à l'échelle, mises à jour sans interruption). C'est pourquoi nous avons utilisé Kubernetes (K8s) pour orchestrer le déploiement de notre service de détection de fraude.

Pour ce projet, nous avons utilisé Minikube, une distribution légère de Kubernetes qui permet de simuler un cluster complet sur une machine locale.

4.1 Définition de l'Architecture

L'infrastructure est définie sous forme de code ("Infrastructure as Code") via des fichiers manifestes YAML situés dans le dossier k8s/. Notre déploiement repose sur trois objets principaux :

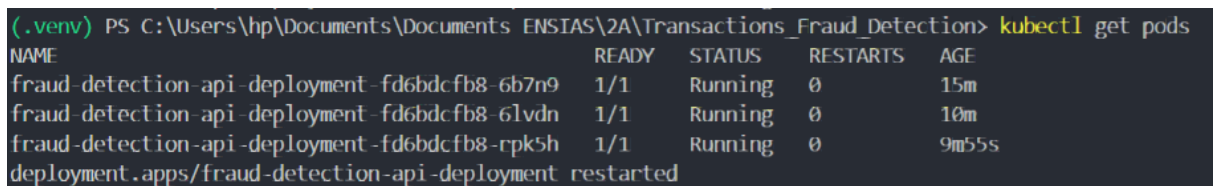
4.1.1 Le Deployment

L'objet Deployment est responsable de maintenir l'application en vie. Nous avons configuré 3 réplicas (Pods) de notre service.

- **Résilience** : Si un Pod plante (erreur interne), Kubernetes le redémarre automatiquement.
- **Scalabilité** : Les 3 instances tournent en parallèle pour absorber la charge.

```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: fraud-detection-api-deployment
5    labels:
6      app: fraud-detection-api-model
7
8  spec:
9    replicas: 3
10   selector:
11     matchLabels:
12       app: fraud-detection-api-model
13   template:
14     metadata:
15       labels:
16         app: fraud-detection-api-model
17     spec:
18       containers:
19       - name: fraud-detection-api-containers
20         image: soufianeeloulji/fraud_detection:latest
21         imagePullPolicy: Always
22         ports:
23         - containerPort: 8000
24         envFrom:
25         - configMapRef:
26           name: fraud-model-config
```

Listing 5 – Code du Deployment - Fichier k8s/deployment.yaml



```
(.venv) PS C:\Users\hp\Documents\Documents ENSIAS\2A\Transactions_Fraud_Detection> kubectl get pods
NAME                                                    READY   STATUS    RESTARTS   AGE
fraud-detection-api-deployment-fd6bdcfb8-6b7n9        1/1     Running   0           15m
fraud-detection-api-deployment-fd6bdcfb8-6lvdn        1/1     Running   0           10m
fraud-detection-api-deployment-fd6bdcfb8-rpk5h        1/1     Running   0           9m55s
deployment.apps/fraud-detection-api-deployment restarted
```

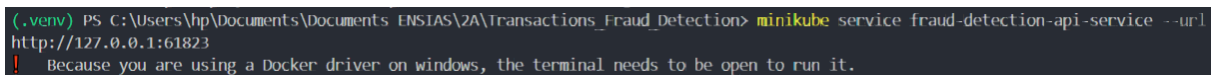
FIGURE 4.1 – Visualisation des 3 Pods actifs (Running)

4.1.2 Le Service

Les Pods étant éphémères (ils peuvent changer d'adresse IP), nous avons créé un objet Service. Il agit comme un point d'entrée stable et répartit le trafic entrant de manière équilibrée entre les 3 répliques disponibles.

```
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: fraud-detection-api-service
5  spec:
6    selector:
7      app: fraud-detection-api-model
8    ports:
9      - protocol: TCP
10        port: 80
11        targetPort: 8000
12    type: NodePort
```

Listing 6 – Code du Service - Fichier k8s/deployment.yaml



```
(.venv) PS C:\Users\hp\Documents\Documents ENSIAS\2A\Transactions_Fraud_Detection> minikube service fraud-detection-api-service --url
http://127.0.0.1:61823
! Because you are using a Docker driver on windows, the terminal needs to be open to run it.
```

FIGURE 4.2 – Obtention de l'URL d'accès au service via Minikube

4.2 Gestion Externalisée de la Configuration

L'une des meilleures pratiques MLOps est de découpler la configuration du code. Nous avons utilisé un ConfigMap pour injecter le seuil de détection de fraude (`FRAUD_THRESHOLD`) directement sous forme de variable d'environnement.

```
1  apiVersion: v1
2  kind: ConfigMap
3  metadata:
4    name: fraud-model-config
5  data:
6    FRAUD_THRESHOLD: "0.90"
```

Listing 7 – Code du ConfigMap - Fichier k8s/configmap.yaml

Cette approche permet aux Data Scientists ou aux administrateurs de modifier la sensibilité du modèle en production par une simple commande `kubectl apply`, sans avoir à toucher au code Python ni à reconstruire l'image Docker.

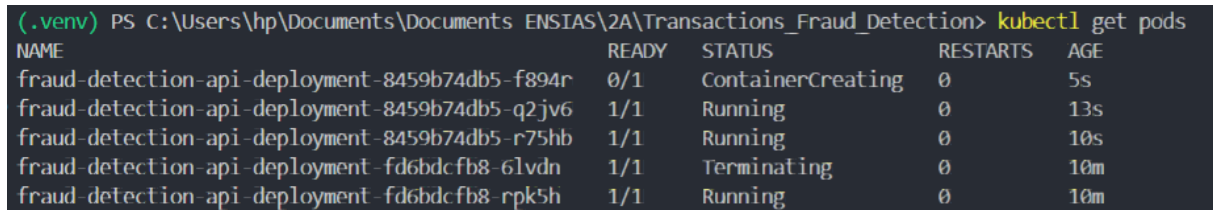
4.3 Stratégie de Mise à Jour (Rolling Update)

Kubernetes nous permet d'effectuer des mises à jour sans interruption de service (Zero Downtime). Lors d'une mise à jour (changement d'image ou de configuration), Kubernetes applique une stratégie de Rolling Update :

1. Il crée un nouveau Pod avec la nouvelle version.
2. Il attend que ce Pod soit "prêt" (Ready).
3. Il termine l'un des anciens Pods
4. Il répète l'opération jusqu'à ce que tous les Pods soient à jour.

Nous avons vérifié ce comportement en modifiant le ConfigMap et en redémarrant le déploiement :

```
kubectl rollout restart deployment fraud-detection-api-deployment
```



```
(.venv) PS C:\Users\hp\Documents\Documents ENSIAS\2A\Transactions_Fraud_Detection> kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
fraud-detection-api-deployment-8459b74db5-f894r	0/1	ContainerCreating	0	5s
fraud-detection-api-deployment-8459b74db5-q2jv6	1/1	Running	0	13s
fraud-detection-api-deployment-8459b74db5-r75hb	1/1	Running	0	10s
fraud-detection-api-deployment-fd6bdcfb8-6lvdn	1/1	Terminating	0	10m
fraud-detection-api-deployment-fd6bdcfb8-rpk5h	1/1	Running	0	10m

FIGURE 4.3 – Démonstration du Rolling Update

Chapitre 5

Validation et Démonstration Fonctionnelle

Une fois l'infrastructure Kubernetes opérationnelle, nous avons procédé à la validation fonctionnelle de bout en bout pour confirmer que le service est accessible et que le modèle effectue correctement ses prédictions.

5.1 Accès à l'Interface de Documentation (Swagger UI)

Nous accédons à une interface interactive via l'URL exposée par le service Kubernetes (ou le tunnel Minikube). Cette interface permet aux développeurs et aux testeurs de visualiser les endpoints disponibles sans consulter le code source.

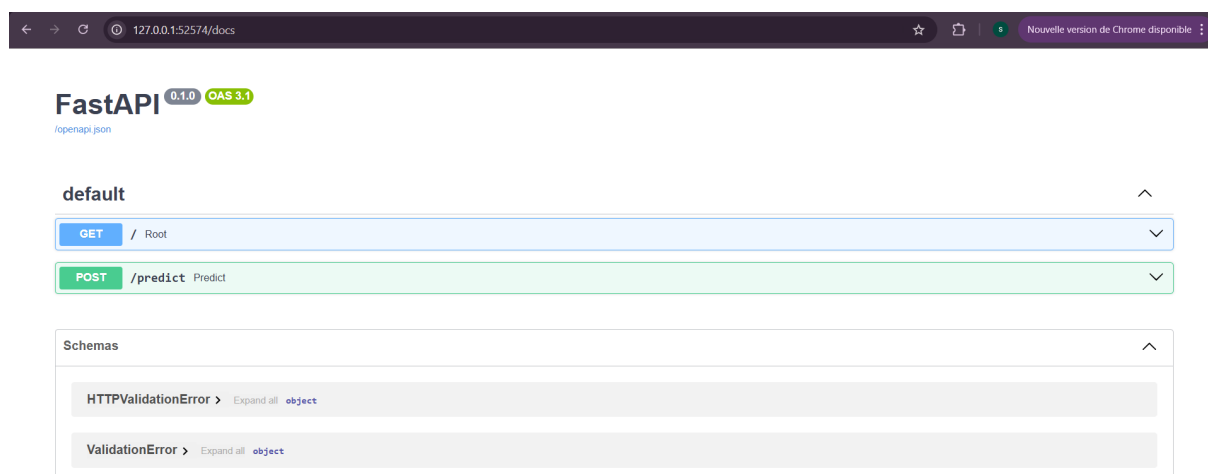


FIGURE 5.1 – Interface listant les endpoints disponibles

5.2 Test de Prédiction

Nous avons simulé une transaction via l'endpoint `/predict` pour valider toute la chaîne de traitement : réception du JSON, prétraitement des données (pipeline Scikit-Learn) et inférence du modèle.

Scénario de test :

- **Entrée :** Une transaction sous forme de fichier json.

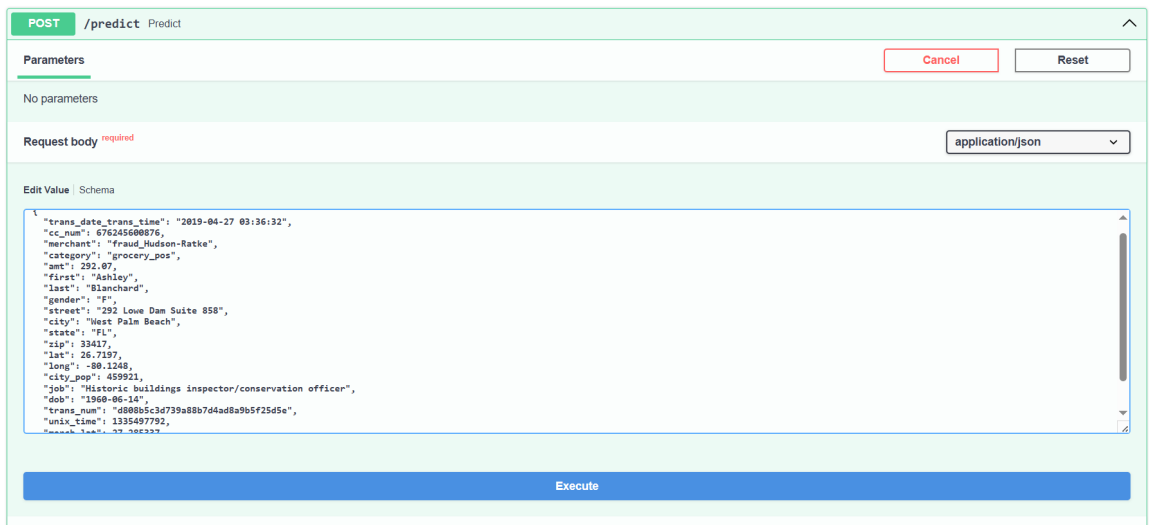


FIGURE 5.2 – Entrée de Test pour l’API

- **Résultat attendu** : Un JSON contenant la probabilité de fraude et le statut booléen.



FIGURE 5.3 – Réponse de l’API

Ce test confirme que notre architecture MLOps est fonctionnelle : le code développé localement a été transporté, déployé et exécuté avec succès dans un environnement de production simulé.

Conclusion Générale

Au terme de ce projet, nous avons réussi à dépasser le stade du simple prototypage pour livrer une application de détection de fraude fonctionnelle.

La mise en place de cette architecture MLOps a permis d'atteindre plusieurs objectifs critiques :

- **La Fiabilité :** Grâce aux tests automatisés intégrés dans le pipeline CI/CD, chaque modification du code est validée avant d'être intégrée, réduisant drastiquement le risque de régression.
- **La Portabilité :** L'utilisation de Docker garantit que le service fonctionne de manière identique sur la machine du développeur et sur le cluster de déploiement.
- **La Scalabilité et la Disponibilité :** L'orchestration par Kubernetes assure que le service reste accessible même en cas de panne d'une instance, tout en permettant une gestion fine des ressources.

En conclusion, ce projet illustre comment l'automatisation et les bonnes pratiques d'ingénierie logicielle transforment un algorithme isolé en un produit industriel robuste et maintenable.

Bibliographie

- [1]FastAPI Documentation. Tiangolo. Disponible sur : <https://fastapi.tiangolo.com/>
- [2]Docker Documentation. Docker Inc. Disponible sur : <https://docs.docker.com/>
- [3]Kubernetes Documentation. The Linux Foundation. Disponible sur : <https://kubernetes.io/docs/>
- [4]GitHub Actions Documentation. GitHub Docs. Disponible sur : <https://docs.github.com/en/actions>
- [5]Scikit-Learn User Guide. Scikit-learn Developers. Disponible sur : https://scikit-learn.org/stable/user_guide.html